Gitthub Code Explanation.

contract SmartRidePool {

↳ it means smart contract named smartridepool is created.

struct Ride {

↳ datatype defined.

uint256 rideId → unique id for each ride

address driver → Etherium address for each ride.

address[ ] passengers → Dynamic array storing passengers.

uint256 farePerPerson → cost per passengers.

uint256 totalLockedAmount → total ETH Locked by passengers, Locked for some period.

uint256 maxpassengers → maximum allowed passengers.

bool started;
bool completed; } → Ride status flags.

mapping(address ⇒ bool) has joined; ⟹ it basically tracks if a user joined the ride.

mapping (address ⇒ bool) has boarded; ⟹ tracks if passengers actually boarded or not.

mapping cannot be returned directly → that's why seperate view function are used Later.

uint256 public ride counter; → counts total ride created.
↳ auto getter function created.

mapping(uint256 ⇒ ride) private rides; ⟹ • maps rideId → ride struct
• stores all rides.

Events (Blockchain Logs)

↳ it tracks help track activity off-chain (UI, frontend).

event RideCreated (Uint256 rideId, address driver);

↳ this is not Logic, just Logs.

CreateRide

function createRide (uint256 _farePerPerson, uint256 _maxpassengers)

public {

ridecounter++; ⟶ increment ride ID.

Ride storage r = rides [ridecounter];

↳ create reference to ~~view~~ new ride in storage

r.rideId = rideCounter;
r.driver = msg.sender;

} assign ride ID.
msg.sender = person calling function ↓ driver.

emit RideCreated (ridecounter, msg.sender);

↳ Emit event.

Join Ride + Lock Payment

function joinride (uint256 _rideId) public payable {

↓ user sends ETH.

Checks

require (!r.started, "Ride already started");

↳ it means after ride started you cannot join.

① require (!r.has joined [msg.sender], "Already joined");

② require (r.passengers.length < r.maxpassengers, "Ride full");

③ require (msg.value == r.fareperperson, "Incorrect fare");

① → prevents duplicate joining.

② → current no. of passengers < max. allowed.

↳ only allow if seat is available.

③ → msg.value ⇒ ETH sent by user.

r.fareperperson ⇒ require fare.

↳ means that user must send exact fare (no less, no more).

Update state

r.passengers.push (msg.sender) → adds this user to the passengers List.

array of passengers adds the current user.

r.hasjoined [msg.sender] = true; ⟹ this user already joined, i.e prevent duplicates.

↳ mapping (address → bool)

r.totalLockedAmount += msg.value → adding the passengers payment to the ride's total fund

emit PassengerJoined (_rideId, msg.sender);

↳ Basically a log to blockchain, Notify that this passenger joined the ride.

Confirm Boarding

```
function confirmBoarding (uint256 _rideId) public {
    require(r.hasJoined [msg.sender], "Not a passenger");
    require (r.started, "ride not started");
    r.hasBoarded [msg.sender] = true;
    emit PassengerBoarded (_rideId, msg.sender);
```

Penalty

```
function appPenalty (uint256 _rideId, address passenger)
    public {
    require (msg.sender == r.driver, "Only driver");
```

→ only driver can apply penalty.

```
    require (r.hasJoined [passenger], "Not a passenger");
```

→ the person actually booked the ride

```
    require (!r.hasBoarded [passenger], "Passenger boarded");
```

→ Penalty only if passenger did not boarded.

```
    uint256 penalty = r.farePerPerson
```

→ if fare = 100 (Let's think 0.0001 = penalty 0.0001).

```
    Payable(r.driver).transfer(Penalty);
```

→ Sending ETH from contract → driver.

r.totalLockedAmount -= penalty;

↳ Reduce from total store amount, that money has now been paid out.

emitPenaltyApplied(passenger, penalty)

↳ Logs :- which passenger was penalized. how much amount.

Ride completing & Payment release.

function completeRide(uint256 rideId) public {

require(msg.sender == r.driver, "only driver");

↳ means that only driver can complete the ride.

require(r.started, "ride not started");

↳ ride must have already started.

require(!r.complete, "Already completed");

↳ Prevents completion twice.

r.complete = true;

↳ Ride is officially finished.

uint256 amount = r.totalLockedAmount;

↳ store total collected ETH in a variable.

payable(r.driver).transfer(amount);

↳ sends all collected money to driver.

emit RideCompleted (_rideId)
emit PaymentRelease (_rideId, amount)

Logs:-
- completion of ride
- Release payment.

View function.

function getpassengers (uint256 _rideId) public view returns (address[] memory).

→ Returns passengers lists.

functions getridedetails (uint256 _rideId)

returns (addr. driver,
uint256 fare,
uint256 totalAmount,
bool started,
bool completed)